

Covers: Linear, Polynomial, Ridge, Lasso, Logistic, Decision Tree, Random Forest, KNN, Naive Bayes, SVM, Gradient Boosting, K-Means, Hierarchical, DBSCAN, HDBSCAN, Apriori, FP-Growth, ECLAT.

Table of Contents

1. Linear Regression
 2. Polynomial Regression
 3. Ridge Regression
 4. Lasso Regression
 5. Logistic Regression
 6. Decision Tree
 7. Random Forest
 8. K Nearest Neighbour (KNN)
 9. Naive Bayes
 10. Support Vector Machine (SVM)
 11. Gradient Boosting
 12. K-Means Clustering
 13. Hierarchical Clustering
 14. DBSCAN
 15. HDBSCAN
 16. Apriori Algorithm
 17. FP-Growth Algorithm
 18. ECLAT Algorithm
-

1. Linear Regression

Usage: Predict continuous numerical values (regression). **Basic Details:** Supervised. Finds best fit straight line using Gradient Descent.

Formula:

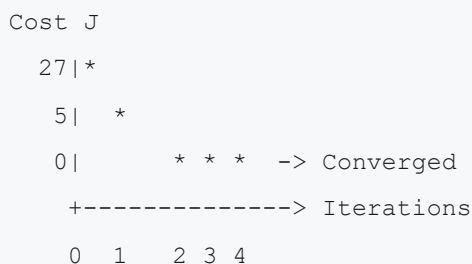
$$y = m \cdot x + b$$

$$\text{MSE (Cost): } J = (1/n) * \text{SUM}((y - y_{\text{pred}})^2)$$

$$\begin{aligned} \text{Gradients: } dJ/dm &= (-2/n) * \text{SUM}(x * (y - y_{\text{pred}})) \\ dJ/db &= (-2/n) * \text{SUM}(y - y_{\text{pred}}) \end{aligned}$$

$$\begin{aligned} \text{Update: } m &= m - \alpha * dJ/dm \\ b &= b - \alpha * dJ/db \end{aligned}$$

Graph:



Iteration Full (Data: $x=[1,2]$, $y=[3,5]$, true line $y=2x+1$, $\alpha=0.1$):

Iter 0: $m=0$, $b=0$

$$\begin{aligned} y_{\text{pred}}(1) &= 0 \cdot 1 + 0 = 0, & \text{error} &= 3 - 0 = 3 \\ y_{\text{pred}}(2) &= 0 \cdot 2 + 0 = 0, & \text{error} &= 5 - 0 = 5 \\ \text{MSE} &= (3^2 + 5^2) / 2 = (9 + 25) / 2 = 17 \\ dJ/dm &= -2/2 * [1 \cdot 3 + 2 \cdot 5] = -(3 + 10) = -13 \\ dJ/db &= -2/2 * [3 + 5] = -8 \\ m_{\text{new}} &= 0 - 0.1 \cdot (-13) = 1.3 \\ b_{\text{new}} &= 0 - 0.1 \cdot (-8) = 0.8 \end{aligned}$$

Iter 1: $m=1.3$, $b=0.8$

$$\begin{aligned} y_{\text{pred}}(1) &= 1.3 \cdot 1 + 0.8 = 2.1, & \text{error} &= 3 - 2.1 = 0.9 \\ y_{\text{pred}}(2) &= 1.3 \cdot 2 + 0.8 = 3.4, & \text{error} &= 5 - 3.4 = 1.6 \\ \text{MSE} &= (0.9^2 + 1.6^2) / 2 = (0.81 + 2.56) / 2 = 1.69 \end{aligned}$$

Iter	m	b	MSE
0	0.00	0.00	17.00
1	1.30	0.80	1.69
2	1.87	0.95	0.12
Final	2.00	1.00	~0

2. Polynomial Regression

Usage: Predict continuous values when relationship is non-linear (curves). **Basic Details:** Supervised. Transforms x into $[x, x^2, x^3 \dots]$ then runs linear regression on new features.

Formula:

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$$

Feature Transform: $x \rightarrow [x, x^2, x^3]$

Graph:



Iteration Full (Fitting $y = x^2$, degree=2):

Step 1 - Feature Transformation:

```
x=1 -> [1, 1^2] = [1, 1]
x=2 -> [2, 2^2] = [2, 4]
x=3 -> [3, 3^2] = [3, 9]
```

Step 2 - Fit linear regression on transformed features ($b_1=0$, $b_2=1$):

```
y_pred(1) = 0*1 + 1*1 = 1    actual=1    error=0
y_pred(2) = 0*2 + 1*4 = 4    actual=4    error=0
y_pred(3) = 0*3 + 1*9 = 9    actual=9    error=0
```

x	x ²	Actual y	Predicted y_pred	Error
1	1	1	1	0
2	4	4	4	0
3	9	9	9	0

MSE = 0 -> perfect fit with degree-2 polynomial.

3. Ridge Regression

Usage: Regression with L2 regularization to prevent overfitting. **Basic Details:** Supervised. Shrinks coefficients towards 0 but never exactly 0.

Formula:

Cost: $J(m) = \text{MSE} + \lambda \cdot \sum(m_j^2)$

Gradient: $dJ/dm_j = d\text{MSE}/dm_j + 2\lambda m_j$

Update: $m_j = m_j - \alpha \cdot (d\text{MSE}/dm_j + 2\lambda m_j)$

Note: $\lambda=0$ -> same as Linear Regression

Large λ -> heavy shrinkage, weights approach 0 but never reach it

Graph:

```
m_j
8 | *
6 |  *
  |   * * * (converges > 0, never hits 0)
  +-----> Penalty lambda
```

Iteration Full ($\lambda=1$, $\alpha=0.1$, MSE gradient=2):

Iter 0: $m_1=8$, $m_2=6$

Penalty $= 1 \cdot (8^2 + 6^2) = 64 + 36 = 100$

Total Cost $J = \text{MSE} + 100 = 200$

$dJ/dm_1 = 2 + 2 \cdot 1 \cdot 8 = 18 \rightarrow m_1 = 8 - 0.1 \cdot 18 = 6.2$

$dJ/dm_2 = 2 + 2 \cdot 1 \cdot 6 = 14 \rightarrow m_2 = 6 - 0.1 \cdot 14 = 4.6$

Iter 1: $m_1=6.2$, $m_2=4.6$

```

Penalty      = 6.2^2 + 4.6^2 = 38.44 + 21.16 = 59.6
dJ/dm1 = 2 + 2*1*6.2 = 14.4  ->  m1 = 6.2 - 0.1*14.4 = 4.76
dJ/dm2 = 2 + 2*1*4.6 = 11.2  ->  m2 = 4.6 - 0.1*11.2 = 3.48

```

Iter	m1	m2	Penalty (SUM m^2)	Total Cost
0	8.0	6.0	100.00	200
1	6.2	4.6	59.60	reduced
2	4.76	3.48	34.73	reduced
Final	~3.5	~2.9	small	converged (never 0)

4. Lasso Regression

Usage: Regression with L1 regularization — drives irrelevant feature weights to exactly 0. **Basic**

Details: Supervised. Uses absolute value penalty; performs automatic feature selection.

Formula:

```

Cost:      J(m) = MSE + lambda * SUM(|mj|)

Gradient: dJ/dmj = dMSE/dmj + lambda * sign(mj)

sign(mj): +1  if mj > 0
          -1  if mj < 0
           0  if mj = 0

Update:    mj = mj - alpha * (dMSE/dmj + lambda * sign(mj))

Note: Small weights hit exactly 0 -> feature is dropped (feature selection)

```

Graph:

```

m_j
8 | *
  | *
  | * <- hits exactly 0 (feature dropped)
  +-----> Penalty lambda

```

Iteration Full (lambda=1, alpha=0.1, MSE gradient=1):

Iter 0: m1=5.5, m2=2.1

```
dJ/dm1 = 1 + 1*sign(5.5) = 1+1 = 2    ->  m1 = 5.5 - 0.1*2 = 5.3
dJ/dm2 = 1 + 1*sign(2.1) = 1+1 = 2    ->  m2 = 2.1 - 0.1*2 = 1.9
```

Iter 1: m1=5.3, m2=1.9

```
dJ/dm1 = 1+1 = 2    ->  m1 = 5.3 - 0.2 = 5.1
dJ/dm2 = 1+1 = 2    ->  m2 = 1.9 - 0.2 = 1.7
```

Iter	m1 (important)	m2 (weak)	Note
0	5.5	2.1	Both active
1	5.3	1.9	Both shrinking
5	4.5	0.5	m2 almost gone
Final	1.8	0.0	m2 dropped! Feature 2 removed

5. Logistic Regression

Usage: Binary Classification (Yes/No, Spam/Not Spam). **Basic Details:** Supervised. Uses Sigmoid curve, outputs probability. Threshold 0.5 -> class label.

Formula:

Linear part: $z = m \cdot x + b$

Sigmoid: $\text{sigma}(z) = 1 / (1 + e^{(-z)})$
Output range: 0 to 1 (probability)

Predict: $\text{class} = 1$ if $\text{sigma}(z) \geq 0.5$
 $\text{class} = 0$ if $\text{sigma}(z) < 0.5$

Loss (Cross Entropy per point):
 $L = -[y \cdot \log(y_{\text{pred}}) + (1-y) \cdot \log(1-y_{\text{pred}})]$

Gradient: $dJ/dm = (1/n) \cdot \text{SUM}(x \cdot (y_{\text{pred}} - y))$

Update: $m = m - \text{alpha} \cdot dJ/dm$

Graph:

```

P
1|      /---
|      /
0|---/
+-----> z

```

Iteration Full (m=1, b=-3, threshold=0.5):

Step 1 - Compute z and sigma(z):

```

x=1:  z = 1*1 + (-3) = -2,  sigma(-2) = 1/(1+e^2) = 1/(1+7.39) = 0.12  -> Pred=0
x=5:  z = 1*5 + (-3) = +2,  sigma(+2) = 1/(1+e^-2) = 1/(1+0.14) = 0.88  -> Pred=1

```

Step 2 - Cross-entropy loss per point:

```

x=1, actual=0:  L = -[0*log(0.12) + 1*log(1-0.12)] = -log(0.88) = 0.13
x=5, actual=1:  L = -[1*log(0.88) + 0*log(0.12)] = -log(0.88) = 0.13

```

Step 3 - Update weights:

```

dJ/dm = (1/2) * [1*(0.12-0) + 5*(0.88-1)] = (1/2)*[0.12 - 0.6] = -0.24
m_new = 1 - 0.1*(-0.24) = 1.024  -> decision boundary sharpens

```

x	Actual	z	sigma(z)	Pred	Correct?
1	0	-2	0.12	0	Yes
3	0	0	0.50	--	border
5	1	+2	0.88	1	Yes

6. Decision Tree

Usage: Classification and Regression. **Basic Details:** Supervised. At each node picks the feature/split giving best Gini or Information Gain.

Formula:

```

Gini(S)      = 1 - SUM( pi^2 )           pi = proportion of class i

Entropy(S)   = - SUM( pi * log2(pi) )

Info Gain    = Entropy(parent) - SUM( |Si|/|S| * Entropy(Si) )

Best split   = feature with highest Info Gain (or lowest Gini)

```

Graph:

```

      [Age < 30?]
      /         \
    [Yes]       [No]
    /           \
  [Buy]         [Don't Buy]

```

Iteration Full (6 samples: 3 Buy, 3 NoBuy — split on Age):

Step 1 - Parent Entropy:

```

P(Buy)=3/6=0.5,    P(NoBuy)=3/6=0.5
Entropy(parent) = -(0.5*log2(0.5) + 0.5*log2(0.5))
                = -(-0.5 + -0.5) = 1.0

```

Step 2 - Split on Age (<30 vs >=30):

```

Left (<30):  3 Buy,  0 NoBuy -> Entropy = -(1*log2(1))      = 0.0 (pure!)
Right (>=30): 0 Buy,  3 NoBuy -> Entropy = -(1*log2(1))      = 0.0 (pure!)
Weighted Entropy = (3/6)*0.0 + (3/6)*0.0 = 0.0
Info Gain(Age)   = 1.0 - 0.0 = 1.0   <- Best split!

```

Split	Left Gini	Right Gini	Weighted Gini	Info Gain
Age	0.00	0.00	0.00	1.0 (Best)
Salary	0.48	0.48	0.48	0.52

7. Random Forest

Usage: Classification and Regression (Ensemble). **Basic Details:** Supervised. Builds multiple decision trees on random bootstrap subsets + random features at each split, then aggregates.

Formula:

```
Classification: Final Class = MODE( T1(x), T2(x), ..., Tn(x) )
Regression:     Final Value = (1/n) * SUM( Ti(x) )
```

Graph:

```
Data -> [Tree 1] -> Yes \
        [Tree 2] -> No   -> Vote = YES (majority)
        [Tree 3] -> Yes /
```

Iteration Full (3 trees, new customer: Age=25, Salary=High):

Bootstrap samples (with replacement):

```
Tree 1 sample: rows [1,1,3,4,5] -> split on Age -> Predict: Yes
Tree 2 sample: rows [2,2,3,4,4] -> split on Salary -> Predict: No
Tree 3 sample: rows [1,2,4,5,5] -> split on Age -> Predict: Yes
```

Vote:

```
Yes: 2 votes, No: 1 vote -> Final = YES
```

Tree	Bootstrap Rows	Feature Used	Prediction
T1	[1,1,3,4,5]	Age	Yes
T2	[2,2,3,4,4]	Salary	No
T3	[1,2,4,5,5]	Age	Yes

Final Vote: 2 Yes vs 1 No -> **YES**

8. K Nearest Neighbour (KNN)

Usage: Classification & Regression based on proximity. **Basic Details:** Supervised. No training phase. At prediction time finds K nearest points and majority votes (classification) or averages (regression).

Formula:

Euclidean Distance:

$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$

Classification: Predict = majority class among K nearest

Regression: Predict = average value among K nearest

Graph:

Y
X (Test)
/ | \
A B C (Nearest neighbors)

Iteration Full (Test Point $T=[2,2]$, $K=3$):

Step 1 - Calculate distances:

$d(T, P1=[1,1]) = \sqrt{(2-1)^2 + (2-1)^2} = \sqrt{1+1} = \sqrt{2} = 1.41$
 $d(T, P2=[2,3]) = \sqrt{(2-2)^2 + (2-3)^2} = \sqrt{0+1} = \sqrt{1} = 1.00$
 $d(T, P3=[5,5]) = \sqrt{(2-5)^2 + (2-5)^2} = \sqrt{9+9} = \sqrt{18} = 4.24$
 $d(T, P4=[3,2]) = \sqrt{(2-3)^2 + (2-2)^2} = \sqrt{1+0} = \sqrt{1} = 1.00$

Step 2 - Sort and pick $K=3$ nearest:

Rank 1: P2 (dist=1.00, Class A)
Rank 2: P4 (dist=1.00, Class B)
Rank 3: P1 (dist=1.41, Class A)
(P3 excluded - too far)

Step 3 - Vote: A=2, B=1 -> Predict: **Class A**

Point	Coords	Distance	In Top 3?	Class
P1	[1,1]	1.41	Yes	A
P2	[2,3]	1.00	Yes	A
P3	[5,5]	4.24	No	B
P4	[3,2]	1.00	Yes	B

9. Naive Bayes

Usage: Classification (Text classification, Spam filtering). **Basic Details:** Supervised. Applies Bayes theorem; assumes features are independent given the class.

Formula:

$\text{Score}(C | X) \propto P(C) * \text{PRODUCT}(P(x_i | C))$

$P(C) = \text{count}(\text{class } C) / \text{total samples}$ [Prior]

$P(x_i | C) = \text{count}(x_i \text{ in } C) / \text{count}(\text{all in } C)$ [Likelihood]

Predict: class with highest score

Laplace Smoothing (fix zero probability):

$P(x_i | C) = (\text{count} + 1) / (\text{total} + \text{num_classes})$

Graph:

$P(\text{Spam}) * P(\text{word} | \text{Spam})$

$P(\text{Spam} | W) = \frac{\text{P(Spam)} * \text{P(word|Spam)}}{\text{P(word)}}$

Iteration Full (Predict: Sunny + No Wind -> Play Tennis?):

Training data (6 rows): Yes=4, No=2

Step 1 - Prior:

$P(\text{Yes}) = 4/6 = 0.667$

$P(\text{No}) = 2/6 = 0.333$

Step 2 - Likelihoods:

$P(\text{Sunny} | \text{Yes}) = 2/4 = 0.50$ (2 of 4 Yes-rows are Sunny)

$P(\text{NoWind} | \text{Yes}) = 4/4 = 1.00$ (all Yes-rows have No Wind)

$P(\text{Sunny} | \text{No}) = 1/2 = 0.50$

$P(\text{NoWind} | \text{No}) = 0/2 = 0.00$ <- zero! use Laplace if needed

Step 3 - Scores:

```

Score(Yes) = P(Yes) * P(Sunny|Yes) * P(NoWind|Yes)
            = 0.667 * 0.50 * 1.00 = 0.333

Score(No)  = P(No) * P(Sunny|No) * P(NoWind|No)
            = 0.333 * 0.50 * 0.00 = 0.000

```

Predict: YES (0.333 > 0.000)

Step	Class Yes	Class No
Prior	0.667	0.333
x P(Sunny/C)	x 0.50	x 0.50
x P(NoWind/C)	x 1.00	x 0.00
Final Score	0.333	0.000

10. Support Vector Machine (SVM)

Usage: Classification and Regression. **Basic Details:** Supervised. Finds the hyperplane that maximizes margin between classes. Support vectors are the closest points to the hyperplane.

Formula:

```

Hyperplane:      w . x + b = 0

Margin:          2 / ||w||      (we want to maximize this)

Objective:       Minimize (1/2) * ||w||^2

Constraint:      yi * (w . xi + b) >= 1   for all points

||w|| = sqrt( w1^2 + w2^2 )
Margin = 2 / sqrt( w1^2 + w2^2 )

```

Graph:

```

Class A      Margin
* *      \      /
*      \ | /      * Class B
        \ | /      * *
          Hyperplane

```

Iteration Full (2D points, finding best w):

Step 1 - Try $w=[1.0, 0.5]$, $b=-1$:

```
||w|| = sqrt(1^2 + 0.5^2) = sqrt(1.25) = 1.12
```

```
Margin = 2 / 1.12 = 1.79
```

```
Constraint check (y=+1 -> score >= 1, y=-1 -> score <= -1):
```

```
Point A=[1,1], y=+1: 1.0*1 + 0.5*1 - 1 = 0.5 (need >= 1 -> violation!)
```

```
Point B=[3,3], y=-1: 1.0*3 + 0.5*3 - 1 = 3.5 (need <= -1 -> violation!)
```

```
Misclassified: 3
```

Step 2 - Optimized $w=[0.8, -0.2]$, $b=0$:

```
||w|| = sqrt(0.8^2 + 0.2^2) = sqrt(0.68) = 0.825
```

```
Margin = 2 / 0.825 = 2.42
```

```
Point A=[1,1], y=+1: 0.8*1 + (-0.2)*1 = 0.6 -> sign correct
```

```
Point B=[3,3], y=-1: 0.8*3 + (-0.2)*3 = 1.8 -> sign correct
```

```
Misclassified: 0 (Best!)
```

Iter	w	Margin	Misclassifications
1	[1.0, 0.5]	1.79	3
2	[0.8, -0.2]	2.42	0 (Best)

11. Gradient Boosting

Usage: Complex Classification & Regression. **Basic Details:** Supervised. Ensemble of weak trees. Each tree fits the residual (error) of all previous trees combined.

Formula:

```
F0(x) = mean(y) [Base prediction]
```

```
ri = yi - F(m-1)(xi) [Residual at step m]
```

```
Fm(x) = F(m-1)(x) + alpha * Tm(x) [Update with new tree]
```

```
Final = F0 + alpha*T1 + alpha*T2 + ...
```

Graph:

```

Pred = Base + alpha*T1 + alpha*T2
100|*
50|  *
0|   * * * (Error approaches 0)
+-----> Trees added

```

Iteration Full (Actual=100, alpha=0.1):

Iter 0 - Base:

```

F0      = mean(y) = 40
r0      = 100 - 40 = +60

```

Iter 1 - Tree 1 predicts residual=60:

```

F1      = F0 + alpha * T1(r0) = 40 + 0.1*60 = 46
r1      = 100 - 46 = +54

```

Iter 2 - Tree 2 predicts residual=54:

```

F2      = F1 + alpha * T2(r1) = 46 + 0.1*54 = 51.4
r2      = 100 - 51.4 = +48.6

```

Iter	Formula	Prediction	Residual
0	$F_0 = 40$ (base)	40	+60
1	$40 + 0.1*60$	46	+54
2	$46 + 0.1*54$	51.4	+48.6
3	$51.4 + 0.1*48.6$	56.3	+43.7
100	...	~100	~0

12. K-Means Clustering

Usage: Partitioning data into K distinct clusters. **Basic Details:** Unsupervised. Requires K upfront. Iterates: assign points to nearest centroid -> recompute centroid.

Formula:

```
Assign: C(x) = argmin_j ||x - mu_j||^2      (nearest centroid)

Update: mu_j = (1/|Cj|) * SUM( x in Cj )    (new centroid = mean)

WCSS:     SUM_j SUM_(x in Cj) ||x - mu_j||^2  (minimize this)
```

Graph:

```

* *           + +
* C1 *        + C2 +
*             + +
```

Iteration Full (6 points, K=2):

Points: A=1, B=2, C=3, D=7, E=8, F=9

Iter 0 - Random centroids: mu1=2, mu2=8

```
d(A=1, mu1=2)=1    d(A=1, mu2=8)=7    -> C1
d(B=2, mu1=2)=0    d(B=2, mu2=8)=6    -> C1
d(C=3, mu1=2)=1    d(C=3, mu2=8)=5    -> C1
d(D=7, mu1=2)=5    d(D=7, mu2=8)=1    -> C2
d(E=8, mu1=2)=6    d(E=8, mu2=8)=0    -> C2
d(F=9, mu1=2)=7    d(F=9, mu2=8)=1    -> C2
```

Iter 1 - Recompute centroids:

```
mu1 = (1+2+3)/3 = 2.0    (unchanged)
mu2 = (7+8+9)/3 = 8.0    (unchanged)
```

Converged after 1 iteration!

Iter	mu1	mu2	Cluster 1	Cluster 2
0	2 (random)	8 (random)	A,B,C	D,E,F
1	2.0	8.0	A,B,C	D,E,F
Final	2.0	8.0	A,B,C	D,E,F

13. Hierarchical Clustering

Usage: Find hierarchical cluster structures without specifying K. **Basic Details:** Unsupervised. Agglomerative (bottom-up). Each point starts as its own cluster. Merge closest pair repeatedly. Cut dendrogram at desired level.

Formula:

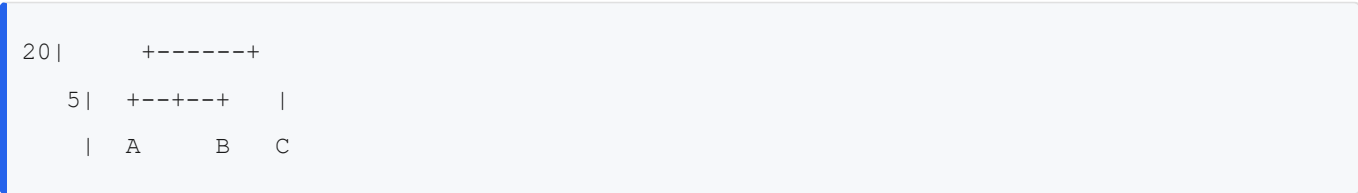
Single Linkage: $d(A,B) = \text{MIN distance between any point in A and any in B}$

Complete Linkage: $d(A,B) = \text{MAX distance between any point in A and any in B}$

Average Linkage: $d(A,B) = \text{MEAN of all pairwise distances}$

Ward Linkage: minimize increase in total within-cluster variance (most common)

Graph (Dendrogram):



Iteration Full (Points: A=1, B=2, C=8, D=9):

Step 1 - Distance matrix:

$d(A,B) = |1-2| = 1$

$d(A,C) = |1-8| = 7$

$d(A,D) = |1-9| = 8$

$d(B,C) = |2-8| = 6$

$d(B,D) = |2-9| = 7$

$d(C,D) = |8-9| = 1$

Step 2 - Merge smallest distances:

Min = $d(A,B)=1$ -> Merge A+B -> Clusters: (A,B), C, D

Min = $d(C,D)=1$ -> Merge C+D -> Clusters: (A,B), (C,D)

Min = $d((A,B), (C,D)) = \min(d(B,C)) = 6$ -> Merge all -> (A,B,C,D)

Iter	Action	Distance	Clusters Remaining
0	Start	--	A, B, C, D
1	Merge A+B	1	(A,B), C, D
2	Merge C+D	1	(A,B), (C,D)
3	Merge all	6	(A,B,C,D)

Cut at height > 1 -> 2 clusters: {A,B} and {C,D}

14. DBSCAN

Usage: Clustering with noise detection (spatial data). **Basic Details:** Unsupervised. No K needed. Labels dense-reachable points as clusters, outliers as noise (-1).

Formula:

```
N_eps(p) = { q | d(p,q) <= eps }           [Neighborhood of p]

Core point:    |N_eps(p)| >= min_samples
Border point:  not core, but within eps of a core point
Noise:         not core and not reachable from any core point -> label = -1
```

Graph:

```
Cluster      Noise
* * *         x
* *
```

Iteration Full (eps=2, min_samples=3, points: A=1, B=2, C=3, D=4, X=10):

Step 1 - Count neighbors within eps=2:

```
N(A=1):  B(dist=1), C(dist=2)           -> 2 neighbors < 3 -> NOT core
N(B=2):  A(dist=1), C(dist=1), D(dist=2) -> 3 neighbors >= 3 -> CORE
N(C=3):  A(dist=2), B(dist=1), D(dist=1) -> 3 neighbors >= 3 -> CORE
N(D=4):  B(dist=2), C(dist=1)           -> 2 neighbors < 3 -> border
N(X=10): none within 2                  -> 0 neighbors -> NOISE
```

Step 2 - Expand cluster from core B:

```
B is core -> start Cluster 1
-> Add A (reachable from B) -> border
-> Add C (reachable from B) -> core, expand
    -> Add D (reachable from C) -> border
Cluster 1 = {A, B, C, D}
X -> Noise = -1
```

Point	Neighbors (eps=2)	Count	Label
A=1	B, C	2	Border (Cluster 1)
B=2	A, C, D	3	Core (Cluster 1)
C=3	A, B, D	3	Core (Cluster 1)
D=4	B, C	2	Border (Cluster 1)
X=10	--	0	Noise (-1)

15. HDBSCAN

Usage: Density clustering with varying densities + automatic noise detection. **Basic Details:** Unsupervised. Extends DBSCAN. Computes MRD -> MST -> hierarchy -> picks stable clusters automatically.

Formula:

```
core_dist_k(p) = distance from p to its k-th nearest neighbor

MRD(a,b) = MAX( core_dist(a), core_dist(b), d(a,b) )

Stability(C) = SUM over p in C of ( 1/lambda_birth(p) - 1/lambda_death(p) )
              lambda = 1/distance (higher lambda = higher density)

Select clusters with highest stability; rest = Noise (-1)
```

Graph:

```
Stable  ||||  <- survives at high density = real cluster
Noise   |  |  <- disappears early = noise (-1)
```

Iteration Full (min_samples=2):

Step 1 - Core distances:

```
Dense region: P1=1, P2=2, P3=3  -> core_dist ~ 1 (neighbors close)
Sparse point: P4=10              -> core_dist ~ 7 (neighbors far)
```

Step 2 - MRD calculation:

```
d(P1,P2)=1,    core(P1)=1, core(P2)=1
MRD(P1,P2) = MAX(1, 1, 1) = 1    <- small, same cluster

d(P3,P4)=7,    core(P3)=1, core(P4)=7
MRD(P3,P4) = MAX(1, 7, 7) = 7    <- large, different cluster
```

Step 3 - Build MST, remove largest edges first:

```
Remove MRD=7 first -> {P1,P2,P3} splits from {P4}
P4 has no stable cluster -> Noise (-1)
```

Pair	core(a)	core(b)	d(a,b)	MRD	Same Cluster?
P1, P2	1	1	1	1	Yes
P2, P3	1	1	1	1	Yes
P3, P4	1	7	7	7	No (noise)

16. Apriori Algorithm

Usage: Market Basket Analysis — find frequent item combinations and association rules. **Basic**

Details: Unsupervised. BFS level-by-level candidate generation. Anti-monotone property: if an itemset is infrequent, all its supersets are pruned.

Formula:

```
Support(X)    = count(transactions containing X) / total transactions

Confidence(X -> Y) = Support(X union Y) / Support(X)

Lift(X -> Y)   = Confidence(X -> Y) / Support(Y)
```

Graph:

```
Level 1: {Milk}, {Bread}, {Butter}          <- scan, count, prune
Level 2: {Milk,Bread}, {Milk,Butter}, {Bread,Butter} <- combine survivors, prune
Level 3: {Milk,Bread,Butter}                  <- pruned if count < min
```

Iteration Full (5 transactions, min_support = 3):

Transactions: T1={M,B,Bu}, T2={B,Bu}, T3={M,B}, T4={M,Bu}, T5={M,B,Bu}

Level 1 - Count singles:

```
{Milk}    = T1,T3,T4,T5 = 4  >=3  keep
{Bread}   = T1,T2,T3,T5 = 4  >=3  keep
{Butter}  = T1,T2,T4,T5 = 4  >=3  keep
```

Level 2 - Count pairs:

```
{Milk,Bread} = T1,T3,T5      = 3  keep
{Milk,Butter} = T1,T4,T5     = 3  keep
{Bread,Butter} = T1,T2,T5    = 3  keep
```

Level 3 - Count triple:

```
{Milk,Bread,Butter} = T1,T5 = 2  < 3  PRUNED
```

Itemset	Count	Support	Kept?
{Milk}	4	0.80	Yes
{Bread}	4	0.80	Yes
{Butter}	4	0.80	Yes
{Milk,Bread}	3	0.60	Yes
{Milk,Butter}	3	0.60	Yes
{Bread,Butter}	3	0.60	Yes
{Milk,Bread,Butter}	2	0.40	No

Rule Example:

```
{Bread} -> {Butter}:
Confidence = Support(Bread,Butter) / Support(Bread) = 0.60 / 0.80 = 0.75
(75% of Bread transactions also contain Butter)
```

17. FP-Growth Algorithm

Usage: Fast Frequent Itemset Mining — faster than Apriori on large datasets. **Basic Details:**

Unsupervised. Compresses transactions into FP-Tree (prefix tree). No candidate generation — only 2 DB scans.

Formula:

Conditional Pattern Base of item i
= all prefix paths in FP-Tree that lead to node i

Support(i in CPB) = sum of counts on those prefix paths

Mine recursively for each item (least frequent first)

Graph:

```

    ROOT
      |
    Milk:4
      /  \
Bread:3 Butter:1
      |
    Butter:2
```

Iteration Full (same 5 transactions, min_support=3):

Step 1 - Frequencies: Milk=4, Bread=4, Butter=4

Step 2 - Build FP-Tree (insert each transaction, sorted by freq):

```
T1 Milk->Bread->Butter:  ROOT->Milk:1->Bread:1->Butter:1
T2 Bread->Butter:         ROOT->Bread:1->Butter:1    (new branch)
T3 Milk->Bread:           ROOT->Milk:2->Bread:2
T4 Milk->Butter:          ROOT->Milk:3->Butter:1      (new branch under Milk)
T5 Milk->Bread->Butter:  ROOT->Milk:4->Bread:3->Butter:2
```

Step 3 - Mine Butter (collect conditional pattern base):

Paths ending at Butter:

[Milk, Bread]	count=2	(from Milk->Bread->Butter path)
[Milk]	count=1	(from Milk->Butter path)
[Bread]	count=1	(from Bread->Butter path)

Conditional freq: Milk = 2+1 = 3 (keep), Bread = 2+1 = 3 (keep)

Patterns with Butter: {Butter}=4, {Milk,Butter}=3, {Bread,Butter}=3

Step	Action	Result
1	Count + sort	Milk=4, Bread=4, Butter=4
2	Insert T1..T5 into tree	FP-Tree built (shared prefixes)
3	Mine Butter's CPB	{Milk,Butter}=3, {Bread,Butter}=3
4	Mine Bread's CPB	{Milk,Bread}=3
Final	All patterns >= 3	6 frequent itemsets

18. ECLAT Algorithm

Usage: Frequent Itemset Mining using vertical TID-set format. **Basic Details:** Unsupervised. Stores item -> {transaction IDs}. Support = size of TID set. Intersect sets for multi-item support. Uses DFS — no repeated DB scans.

Formula:

```
Support(A)      = |TID(A)|      (size of TID set)

Support(A u B) = |TID(A) intersect TID(B)| (size of intersection)

Keep if support >= min_support
```

Graph:

```
Milk:  {T1, T3, T4, T5}
Bread: {T1, T2, T3, T5}
Milk intersect Bread = {T1, T3, T5} -> Support = 3
```

Iteration Full (same 5 transactions, min_support=3):

Step 1 - Build vertical format:

```
Milk  -> {T1, T3, T4, T5}  size=4  keep
Bread -> {T1, T2, T3, T5}  size=4  keep
Butter -> {T1, T2, T4, T5} size=4  keep
```

Step 2 - 2-itemset intersections:

$\{\text{Milk}, \text{Bread}\} = \{T1, T3, T4, T5\} \cap \{T1, T2, T3, T5\} = \{T1, T3, T5\} \quad \text{size}=3 \quad \text{keep}$
 $\{\text{Milk}, \text{Butter}\} = \{T1, T3, T4, T5\} \cap \{T1, T2, T4, T5\} = \{T1, T4, T5\} \quad \text{size}=3 \quad \text{keep}$
 $\{\text{Bread}, \text{Butter}\} = \{T1, T2, T3, T5\} \cap \{T1, T2, T4, T5\} = \{T1, T2, T5\} \quad \text{size}=3 \quad \text{keep}$

Step 3 - 3-itemset intersection:

$\{\text{Milk}, \text{Bread}, \text{Butter}\} = \{T1, T3, T5\} \cap \{T1, T2, T4, T5\} = \{T1, T5\} \quad \text{size}=2 \quad \text{PRUNED}$

Itemset	TID Set	Support	Valid?
{Milk}	{T1,T3,T4,T5}	4	Yes
{Bread}	{T1,T2,T3,T5}	4	Yes
{Butter}	{T1,T2,T4,T5}	4	Yes
{Milk,Bread}	{T1,T3,T5}	3	Yes
{Milk,Butter}	{T1,T4,T5}	3	Yes
{Bread,Butter}	{T1,T2,T5}	3	Yes
{Milk,Bread,Butter}	{T1,T5}	2	No